FIAS Frankfurt Institute for Advanced Studies

GOETHE UNIVERSITÄT FRANKFURT AM MAIN

High-Performance Computer Architectures Practical Course

Compute Unified Device Architecture (CUDA)

Prof. Dr. Ivan Kisel
Robin Lakos
Akhil Mithran
Oddharak Tyagi

June 24, 2025

Introduction

- Main components in GPU programming:
  - CPU (processor),
  - RAM (main memory), and
  - GPU (graphics card) with its own VRAM (Video RAM).
- All components connect via the mainboard.
- The CPU and RAM use dedicated sockets; a dedicated GPU connects through a PCIe slot that can be used for different types of hardware (graphics cards, network adapters, sound cards, ...).
- PCIe bandwidth is crucial for GPU performance and can be a bottleneck for data transfer, which itself is already time costly.
- The GPU contains many thousands of computation units for parallel processing.

GP-GPU Programming

Historically, GPUs were used for – as the same suggests – graphics (e.g. translation, rotation, scaling, ... of objects), where matrix or tensor computations are the key problem to solve in an efficient way.

- General-Purpose GPU (GP-GPU) programming means using graphics processors for tasks beyond graphics rendering.
- GPUs are optimized for parallel, high-throughput computations such as those found in graphics, which makes them highly suitable for scientific and data workloads.
- Modern frameworks like Nvidia CUDA and AMD HIP have made GPU programming more accessible, offering programmer-friendly APIs and robust toolchains.
- These frameworks enable efficient parallel programming on GPUs and are widely used in fields such as scientific computing, machine learning, and data analysis.
- CUDA and HIP are device-specific: CUDA works only on Nvidia GPUs, HIP primarily on AMD hardware, which limits code portability.
- OpenCL, in contrast, is an open standard supporting various devices (GPUs, CPUs, FPGAs), but often requires more boilerplate and may lack advanced features found in CUDA or HIP.
- Framework choice depends on hardware targets, desired performance, and portability needs.

GPU processing units are to some extend very ”basic” or ”limited” processors, but they are very efficient in things they can do.

Workflow

The general workflow of performing calculations on GPUs is as follows:

1. Initialize data on CPU
2. Copy data from CPU to GPU memory (from RAM to VRAM)
3. Launch GPU calculations
4. In the ideal case, run independent work on CPU to keep high load on all devices
5. Synchronize GPU
6. Copy data from GPU to CPU (from VRAM back to RAM)
7. Post-process data on CPU

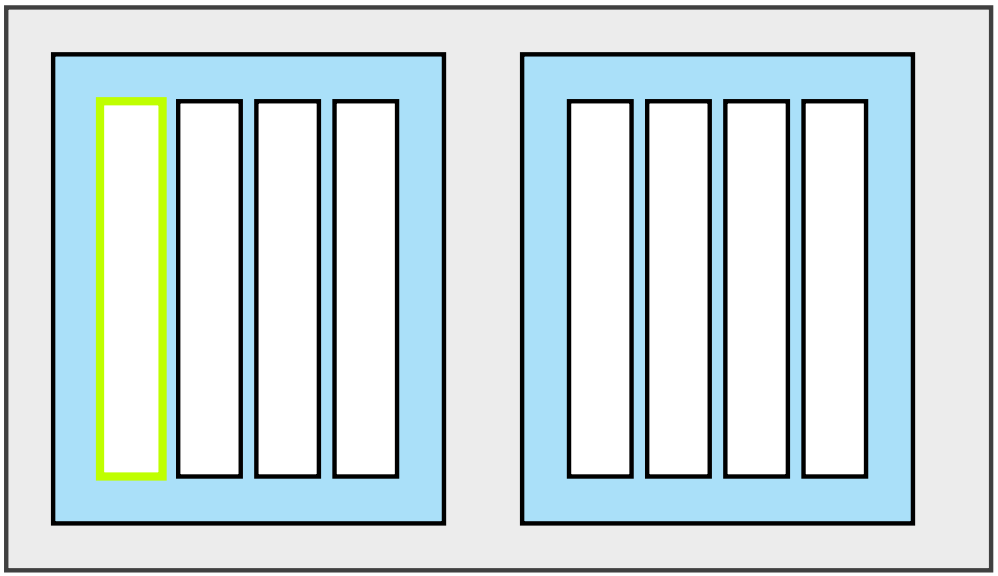
Nvidia GPUs

CUDA Execution Hierarchy:

- **Grid:** A collection of blocks that executes a kernel on the GPU. One grid is mapped to one GPU.
- **Block:** A group of threads executed together on a single Streaming Multiprocessor (SM).
- **Thread:** The smallest unit of execution. Each thread runs on a single core within an SM. The maximum number of threads per block is 1024.
- **Warp:** A group of (usually) 32 threads within a block that execute instructions in lockstep (all threads perform the same instruction at the same time). If threads take different code paths (branching), this leads to branch divergence and lower efficiency.

GPU Hierarchy

```
performWork<<<2, 4>>>()
```



The figure visualizes the launch of a **kernel** (function) with a **execution configuration** (inside the "<<<" and ">>>") such that it executes on a grid with 2 blocks with 4 threads per block (one highlighted in green) each.

Each block in a grid contains the same number of threads.

Writing a GPU Function (Kernel) in C++/CUDA

```
void CPUFunction()
{
  printf("This function is defined to run on the CPU.\n");
}
__global__ void GPUFunction()
{
  printf("This function is defined to run on the GPU.\n");
}
int main()
{
  CPUFunction();
  GPUFunction<<<1, 1>>>(); // execution configuration: 1 block, 1 thread per block
  cudaDeviceSynchronize();
}
```

The **`__global__`** keyword indicates that the following function will run on the GPU, and can be invoked *globally*, which in this context means either by the CPU, or, by the GPU.

Please note that there is no return type allowed, as the GPU executes on pre-allocated memory only.

Launching kernels is asynchronous! Therefore, `cudaDeviceSynchronize()` is crucial to instruct the CPU to wait for the GPU code to complete. Before completion, do not try to read out the VRAM.

The kernel code is executed by every thread in every block configured.

CUDA Function and Variable Definitions (1)

CUDA allows to read out hardware information from inside the C++ source code.

Examples:

```
int deviceId;
cudaGetDevice(&deviceId);

cudaDeviceProp deviceProp;
cudaGetDeviceProperties(&deviceProp, deviceId);

int computeCapabilityMajor = deviceProp.major;
int computeCapabilityMinor = deviceProp.minor;
int multiProcessorCount = deviceProp.multiProcessorCount;
int warpSize = deviceProp.warpSize;
```

CUDA Function and Variable Definitions (2)

Important variables for mapping data to threads in kernel functions:

```
gridDim.x  // number of blocks
blockIdx.x  // index of the current block within the grid
blockDim.x  // number of threads in a block
threadIdx.x  // index of a thread within a block

dim3 n_threads(16, 16, 1); // a 16 x 16 block threads
dim3 n_blocks((N / n_threads.x) + 1, (N / n_threads.y) + 1, 1);
kernel<<<n_blocks, n_threads>>>(...)
// then, inside kernel:
...
int row = blockIdx.x * blockDim.x + threadIdx.x;
int col = blockIdx.y * blockDim.y + threadIdx.y;
...
```

Example:

Assume an execution configuration of `<<<2, 4>>>`. Mapping each thread to an array of size $2 * 4 = 8$ with elements $0, 1, \ldots, 7$ can be done by using :

$$\texttt{globalIdx} = \texttt{threadIdx.x} + \texttt{blockIdx.x} * \texttt{blockDim.x}$$

This is similar to what we have done previously when calculating indices from 1D to 2D. The formula can be extended analog for ND.

But what if the number of elements exceeds the total number of available threads? $\rightarrow$ **Grid-stride loop**

Grid-Stride Loops

When the number of elements (e.g. in a matrix) exceeds the number of total threads set up in the execution configuration, **grid-stride loops** help to map the data to the available threads.

Example: A function that multiplies all elements by 2.

```
__global__
void doubleElements(int *a, int N)
{
  int indexWithinTheGrid = threadIdx.x + blockIdx.x * blockDim.x;
  int gridStride = gridDim.x * blockDim.x;  // total number of threads to shift

  for (int i = indexWithinTheGrid; i < N; i += gridStride)
  {
    a[i] *= 2;
  }
}
```

Here, each thread jumps through the array (based on the stride) and processes several array elements.

In this example, we added to work with data on the GPU. However, since the GPU is actually a different device, one has to move the data to that device. But, transferring data to the GPU like it is needed here also requires to allocate memory on the device first, as we have to move it to a defined place.

Memory Allocation in CUDA (1)

```
int main()
{
  int N = 10000;
  int *a;
  size_t size = N * sizeof(int);
  cudaMallocManaged(&a, size);  // allocation of unified (!) memory
  init(a, N); // e.g. by mapping a[i] = i
  size_t n_threads = 256;
  size_t n_blocks = 32;
  // Note: the size of this grid is 256*32 = 8192 < 10000 (= N).
  doubleElements<<<n_blocks, n_threads>>>(a, N);
  cudaDeviceSynchronize();
  cudaFree(a); // free allocated unified memory
}
```

The CUDA function `cudaMallocManaged` allocates unified memory that is accessible via CPU and GPU. The function is blocking (waits until allocation is done). Here, `cudaMemPrefetchAsync` *can* be used to hint the CUDA runtime to transfer the data at some point. Unified memory is convenient but may have performance overhead compared to explicit management.

CUDA also allows to allocate memory more specific, e.g. by using `cudaMalloc` (memory allocation on device) or `cudaMallocHost` (memory allocation on host). However, this also requires to use for instance `cudaMemcpy` to copy the data from one memory buffer into another, e.g. from RAM into VRAM.

Memory Allocation in CUDA (2)

Example: Manually Managed Memory Allocation

```
int size = 100'000;

double *src, *dest;
cudaMallocHost(&src, sizeof(double) * size);
cudaMallocHost(&dest, sizeof(double) * size);
double *d_src, *d_dest;
cudaMalloc(&d_src, sizeof(double) * size);
cudaMalloc(&d_dest, sizeof(double) * size);

cudaMemcpy(d_src, src, sizeof(double) * size, cudaMemcpyHostToDevice);

int n_blocks = 64;
int n_threads = 32;
kernel<<<n_blocks, n_threads>>>(d_src, d_dest, size);

cudaDeviceSynchronize();

cudaMemcpy(dest, d_dest, sizeof(double) * size, cudaMemcpyDeviceToHost);

cudaFree(d_src);
cudaFree(d_dest);
cudaFreeHost(src);
cudaFreeHost(dest);
```

CUDA Streams

Launching kernels as shown- before will always launch these kernel functions using the default stream. All executions in the default stream are serialized, thus, their execution is in order, e.g. when multiple kernels are launched into the default stream (stream 0).

In general, a stream is a series of operations executed in issue-order.

To overlap the processing on GPU, it is possible to create multiple streams. In each stream, the execution is in order, but these streams can run in parallel (if hardware is ”free”) and, thus, no ordering is guaranteed between operations issued in different non-default streams.

Operations in the default stream can *not* execute at the same time as operations in any non-default stream (on modern CUDA, this is relaxed, see ”legacy default stream behavior”). However, kernel launches and many other CUDA runtime operations are run by default in the default stream.

There are a few rules, concerning the behavior of CUDA streams, that should be learned in order to utilize them effectively:

- Operations within a given stream occur in order.
- Operations in different non-default streams are not guaranteed to operate in any specific order relative to each other.
- The default stream is blocking and will both wait for all other streams to complete before running, and, will block other streams from running until it completes.

Creation of CUDA Streams

```
// Create a stream:
cudaStream_t stream;
cudaStreamCreate(&stream);

// memory allocation / data initialization
...

// launch a kernel into non-default stream:
kernel<<<grid, blocks, 0, stream>>>();

// cleanup
cudaStreamDestroy(stream);
```

Asynchronous Memory Copies in Non-Default Streams (1)

In the ideal case, we would not only like perform calculations in parallel, but also transfer data asynchronously, such that the data transfer can be overlapped with calculations of other streams.

In order to asynchronously copy data, CUDA needs to make assumptions about its location.

Typical host memory uses paging so that in addition to RAM, data can be stored on some backup storage device like a physical disk.

Pinning, or page-locking memory bypasses host OS paging, storing allocated memory in RAM. Page-locking, or pinning memory is required to transfer memory asynchronously in a non-default stream.

Because it prevents storage of data on some backup storage, pinned memory is a limited resource, and care must be taken not to over use it.

Pinned memory can be allocated by using `cudaMallocHost`.

```
const uint64_t num_entries = 1UL << 26;
uint64_t *data_cpu;
cudaMallocHost(&data_cpu, sizeof(uint64_t)*num_entries);
```

Asynchronous Memory Copies in Non-Default Streams (2)

To transfer data between host and device asynchronously, `cudaMemcpyAsync` can be used.

```
cudaStream_t stream;
cudaStreamCreate(&stream);
const uint64_t num_entries = 1UL << 26;
uint64_t *data_cpu, *data_gpu;
cudaMallocHost(&data_cpu, sizeof(uint64_t)*num_entries);
cudaMalloc(&data_gpu, sizeof(uint64_t)*num_entries);
cudaMemcpyAsync(data_gpu,
                data_cpu,
                sizeof(uint64_t)*num_entries,
                cudaMemcpyHostToDevice,
                stream);

// do something
...

cudaMemcpyAsync(data_cpu,
                data_gpu,
                sizeof(uint64_t)*num_entries,
                cudaMemcpyDeviceToHost,
                stream);

cudaStreamSynchronize(stream); // can be used to wait until data is fully transferred.
```

Error Handling

Similar to OpenCL, it is helpful to create a function for error handling and debugging:

```
inline cudaError_t checkCuda(cudaError_t result)
{
  if (result ≠ cudaSuccess) {
    fprintf(stderr, "CUDA Runtime Error: %s\n", cudaGetErrorString(result));
    assert(result == cudaSuccess); // requires #include <assert.h>
  }
  return result;
}
```

This function can be applied for return statements of functions to check for errors.

```
// check errors like this:
cudaError_t err;
err = cudaMallocManaged(&a, N);
checkCuda(err);

doubleElements<<<n_blocks, n_threads>>>(a, N);

// or check errors like this:
checkCuda(cudaDeviceSynchronize());

// or this:
checkCuda(cudaGetLastError()); // cudaGetLastError will return the error from above.
```

Multi-GPU Programming

On the machines on side, multi-GPU programming is not possible and neither asked to be done by you. It's just for providing some additional information here.

CUDA allows quite easily to make use of multiple GPUs. The easiest way of doing that is by just looping over all available devices.

```
int num_gpus;
cudaGetDeviceCount(&num_gpus);

for (int gpu = 0; gpu < num_gpus; gpu++) {

    cudaSetDevice(gpu);

    // Perform operations for this GPU.
    ...
}
```

Similar to work with multiple streams, data can be split into chunks that are then processed by different GPUs in parallel. Of course, each GPU can then process multiple streams and each stream can have a grid with several blocks and threads.

Remember to first allocate memory for the host system and all GPUs. Additionally, data has to be transfered to and from the specific devices' memory.

A full example of this can be found in the script.

Optimizing Parameters

The number of GPUs, streams, blocks, and threads, can have a huge impact on the runtime performance of an application.

Some rules of thumb that can be used to start with:

- Number of blocks: Launch enough blocks such that all SMs are fully used. Launch at least as many blocks as SMs, often several times more to keep all SMs busy while others wait (hide memory latency). For example, if you have 22 SMs, launching `n_blocks` = 32 * `n_SMs` is a solid choice to start with.
- Number of threads: Make the number of threads per block a multiple of warp size (usually 32), e.g. 128, 256, 512. This ensures full utilization of warps and avoids partial warps (which are inefficient).

The compute capabilities can give a hint about features that are available on the specific architecture of the GPU.

Optimizing CUDA Code in General

On one hand, optimal parameters can improve the runtime drastically, but compared to efficient programming in general, they can play a minor role.

Nvidia NSight Systems can be used to profile your CUDA application. It will show you exactly where the time is spent (e.g. data transfer, calculations on kernels, in streams, etc.). You can use it and try to overlap as much work as possible.

How to profile your application:

- Run your application under the profiler:
  `nsys profile -o my_profile ./my_executable`
- Open the generated file (`my_profile.qdrep`) in the NSight Systems GUI for detailed analysis.
- Look for time spent in kernels, memory transfers, and idle periods.
- (Optional) Use NVTX markers in code to annotate phases:

```
#include <nvtx3/nvToolsExt.h>
nvtxRangePushA("MyPhase");
// code
nvtxRangePop();
```

The best case in HPC is when every piece of hardware is constantly fully utilized. Therefore, as a good programmer, you can make use of these tools to see where your idle times are and maybe they give a hint on how to fix them.

Common Mistakes & Troubleshooting

- *Forgetting to check for errors:* Always call `cudaGetLastError()` after kernel launches to catch asynchronous errors.
- *Missing `cudaDeviceSynchronize()`:* Remember to synchronize the device before accessing results on the CPU, or when measuring timing.
- *Accessing out-of-bounds memory:* Double-check thread/block indices and array bounds in kernels.
- *Insufficient blocks/threads:* Make sure to launch enough blocks and threads to cover all your data.
- *Resource overuse:* Too much shared memory or too many registers per thread can limit occupancy.
- *Data not copied:* Don't forget explicit `cudaMemcpy` when using `cudaMalloc` (device memory).
- *Incorrect pointer usage:* Host pointers vs. device pointers—ensure you pass the correct type to CUDA functions and kernels.
- *Uninitialized memory:* Always initialize memory before use.
- *Not using pinned memory for async transfers:* Use `cudaMallocHost` for asynchronous `cudaMemcpyAsync`.